

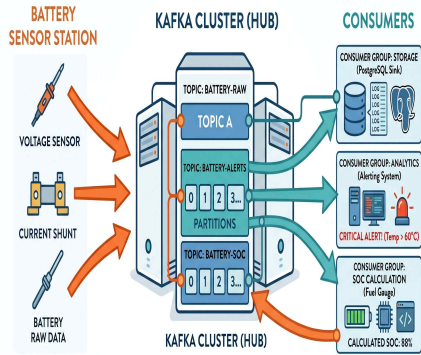
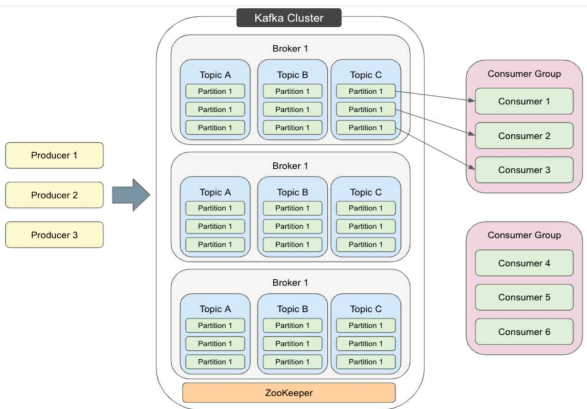
Hanium Project Study

BMS 데이터 파이프라인 실습 내용 공유

1. Apache Kafka (수집 -> 전달) 2. PostgreSQL (전달->저장)

발표자: 김근우 (데이터 추출 담당)

토픽 생성 -> console producer->console consumer



PART 1. Topic

With Apache Kafka

1. 카프카 브로커 실행 옵션 설정(모드 선택)

- 카프카 엔진을 어떤 모드로, 어떤 설정판(설정 파일)을 읽어서 켤까요?"를 지정하는 단계
- `bin/kafka-server-start.sh config/server.properties`
 - >포어그라운드 실행(원격제어) : 노트북의 파워셀 창을 닫거나 컴퓨터가 잠들면 EC2 안의 카프카 서버도 같이 꺼져 버림
- `bin/kafka-server-start.sh -daemon config/server.properties`
 - >백그라운드 실행 : 노트북을 끄고 파워셀을 닫아도 AWS 클라우드 안에서 카프카가 24시간 내내 계속 살아있음
(실전프로젝트에서 사용, 단 비용 청구 주의)

1. 카프카 브로커 실행 옵션 설정 (데이터 받기 설정)

- 카프카 엔진을 어떤 모드로, 어떤 설정판(설정 파일)을 읽어서 켜겠냐?"를 지정하는 단계
- `bin/kafka-server-start.sh config/server.properties`
- -> 명령어 맨 뒤에 붙는 `config/server.properties` 파일은 카프카 브로커의 종합 설정 서류. 이 파일 안의 내용도 카프카가 실행될 때 참고하는 중요한 옵션
- `listeners` 또는 `advertised.listeners` 설정: 카프카 브로커가 외부(내 노트북의 파이썬 코드)로부터 접속을 받을 때 사용할 IP 주소(AWS 퍼블릭 IP)를 적어주는 옵션. 이 부분이 기본값(`localhost`)으로 되어 있으면 외부에서 데이터를 보냈을 때 못 받음.

listeners 또는 advertised.listeners 설정

1. 카프카 폴더 내의 설정 파일을 편집기로 연다
(nano config/server.properties)
2. 화살표 키로 아래로 쪽 내리다 보면 #advertised.listeners=PLAINTEXT://your.host.name:9092
라고 적힌 부분을 찾음. (보통 30~40번째 줄 근처)
3. 앞의 주석(#)을 지우고, your.host.name 자리에 EC2 퍼블릭 IP를 정확히 적어줌
4. Ctrl + O 누르고 엔터 (저장), Ctrl + X (종료)를 눌러 나온다
5. 카프카 서버를 껐다가 다시 켵니다. (Ctrl + C 후 다시 실행 명령어 입력)

2. Topic 생성

```
ec2-user@ip-172-31-58-166: ~$ ssh -i "hanium-final-key.pem" ec2-user@3.38.137.121
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

새로운 기능 및 개선 사항에 대한 최신 PowerShell을 설치하세요! https://aka.ms/PSWindows

PS C:\Users\김근우> cd ~\Downloads
PS C:\Users\김근우\Downloads> ssh -i "hanium-final-key.pem" ec2-user@3.38.137.121
#_
~\_ ##### Amazon Linux 2023
#####
#####
##### \###|
##### \|/ --- https://aws.amazon.com/linux/amazon-linux-2023
##### V#| '->
##### /
##### .-./
##### _/_/_/_/_/
##### _/n/'

Last login: Sun May 17 12:23:07 2026 from 155.230.85.42
[ec2-user@ip-172-31-58-166 ~]$ cd kafka_2.13-4.2.0
[ec2-user@ip-172-31-58-166 kafka_2.13-4.2.0]$ bin/kafka-topics.sh --create --topic test-topic --bootstrap-server localhost:9092 --partitions 1 --replication-factor 1
Created topic test-topic.
[ec2-user@ip-172-31-58-166 kafka_2.13-4.2.0]$ bin/kafka-topics.sh --list --bootstrap-server localhost:9092
test-topic
[ec2-user@ip-172-31-58-166 kafka_2.13-4.2.0]$
```

1. **creat** : 토픽 생성 명령어
2. **test-topic** : 토픽이름
3. **bootstrap-server** : 토픽 생성할 브로커들의 IP주소 적기
4. **partition** : 파티션 개수를 지정(최소 1개)
5. **replication** : 토픽의 파티션 복제개수(0개)
(파티션의 데이터는 각 브로커마다 저장)

(토픽 상태 확인)

```
[ec2-user@ip-172-31-58-166 kafka_2.13-4.2.0]$ bin/kafka-topics.sh --describe --topic test-topic --bootstrap-server localhost:9092
Topic: test-topic      TopicId: cbaXLDikSKmWwghi5I42CA PartitionCount: 1      ReplicationFactor: 1    Configs: min.insync.replicas=1,segment.bytes=1073741824
      Topic: test-topic Partition: 0    Leader: 1      Replicas: 1      Isr: 1 Elr:      LastKnownElr:
[ec2-user@ip-172-31-58-166 kafka_2.13-4.2.0]$
```

describe : 토픽의 상태를 확인 명령어

1. 파티션의 개수(1개,복제0개인 상태)
2. Partition: 0 = 지금 파티션을 1개만 만들었으니, 0번 방 딱 하나만 존재
3. Leader: 1 = 서버의 고유 번호(ID)가 1번이라는 뜻
4. Replicas: 1 & Isr: 1 = 이 파티션의 복사본을 저장/동기화중인게 1번 서버뿐

(토픽 옵션 수정: 파티션 갯수, 리텐션 시간)

```
[ec2-user@ip-172-31-58-166 kafka_2.13-4.2.0]$ bin/kafka-topics.sh --bootstrap-server localhost:9092 --alter --topic test-topic --partitions 3
[ec2-user@ip-172-31-58-166 kafka_2.13-4.2.0]$ bin/kafka-topics.sh --bootstrap-server localhost:9092 --describe --topic test-topic
Topic: test-topic      TopicId: cbaXLDikSKmWWghi5I42CA PartitionCount: 3      ReplicationFactor: 1    Configs: min.insync.replicas=1,segment.bytes=1073741824
Topic: test-topic      Partition: 0    Leader: 1      Replicas: 1      Isr: 1  Elr:      LastKnownElr:
Topic: test-topic      Partition: 1    Leader: 1      Replicas: 1      Isr: 1  Elr:      LastKnownElr:
Topic: test-topic      Partition: 2    Leader: 1      Replicas: 1      Isr: 1  Elr:      LastKnownElr:
```

alter : 파티션 개수 변경 명령어

파티션은 '늘리기'만 가능하고, '줄이기'는 불가능

```
bin/kafka-configs.sh --bootstrap-server localhost:9092 --entity-type topics --entity-name test-topic --alter --add-config retention.ms=3600000(1시간)
```

PART 2. console- producer(consumer)

With Apache Kafka

1.producer -> topic(값만 쏘아보기)

. 토픽에 넣는 데이터 = 레코드(record)

-> 메시지 키(key)와 메시지 값(value)으로 이루어져 있음

1. 콘솔 프로듀서 실행하기 (메시지값만 쏘기)

메시지 키는 자바의 null로 기본 설정되어 브로커로 전송됨.

```
[ec2-user@ip-172-31-58-166 kafka_2.13-4.2.0]$ bin/kafka-console-producer.sh --bootstrap-server localhost:9092 --topic test-topic
>hello
>kafka
>battery
```

2. 콘솔 컨슈머 실행하기 (내가 보낸 메시지 실시간으로 확인하기)

프로듀서는 데이터를 보낼 때 (Round-Robin, 번갈아가며 넣기) 방식 작동

```
[ec2-user@ip-172-31-58-166 kafka_2.13-4.2.0]$ bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic test-topic --from-beginning
hello
kafka
battery
```

1.producer -> topic(값만 쏘아보기)

1. 키(Key) 옵션을 켜고 콘솔 프로듀서 실행하기

-> **parse.key**를 **true**로 두면 레코드를 전송할 때 메시지 키를 추가가능/
키와 값을 구분위해 콜론(:)을 분리 기호로 쓰겠다고 설정 (**key.separator=:**)

```
>bin/kafka-console-producer.sh --bootstrap-server localhost:9092 --topic test-topic --reader-property parse.key=true --r
eader-property key.separator=:
>cell_A:4.20
>cell_B:4.10
>cell_c:4.00
>cell_d:5.00
>cell_e:6.00
```

2.키와 값을 함께 확인하기 (콘솔 컨슈머)

```
[ec2-user@ip-172-31-58-166 kafka_2.13-4.2.0]$ bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic te
st-topic --from-beginning --formatter-property print.key=true --formatter-property key.separator=:
bin/kafka-console-producer.sh --bootstrap-server localhost:9092 --topic test-topic --reader-property parse.key=true --re
ader-property key.separator=:
cell_e:6.00
cell_d:5.00
null:hello
null:kafka
null:battery
cell_A:4.20
cell_B:4.10
cell_c:4.00
```

'해시(Hash) 알고리즘'이 중요한이유

키(Key)를 지정하는 순간, 카프카는 무조건 '해시(Hash) 알고리즘'을 사용해 파티션을 결정(cell_A라는 글자를 해시 함수에 넣음 -> 1 나옴 -> cell_A가 붙은 모든 메시지는 무조건 1번 파티션 방으로만 들어감)

키를 cell_A로 딱 고정해 주면, 수천만 개의 데이터가 들어와도 cell_A 데이터만큼은 무조건 지정된 하나의 파티션 방에만 순서대로 차곡차곡 쌓이게 됨.

덕분에 컨슈머(PostgreSQL에 저장하는 파이썬 코드)가 데이터를 읽어올 때 순서가 절대 뒤바뀌지 않고 안전하게 처리할 수 있음

파티션을 추가하더라도 이전 사용하던 메시지 키의 일관성 보장을 보장하고 싶으면 커스텀 파티셔너를 만들어서 운영해야 함

PART 3. Postgresql

with Apache Kafka

1.AWS EC2에 PostgreSQL 설치하기 (EC2 창)

사용 중인 서버 환경에 맞춰 최신 PostgreSQL 15 버전을 설치하고 가동

```
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing      : 
  Installing     : postgresql15-private-libs-15.16-1.amzn2023.0.1.x86_64
  Installing     : postgresql15-15.16-1.amzn2023.0.1.x86_64
  Installing     : libicu-67.1-7.amzn2023.0.4.x86_64
  Running scriptlet: postgresql15-server-15.16-1.amzn2023.0.1.x86_64
  Installing     : postgresql15-server-15.16-1.amzn2023.0.1.x86_64
  Running scriptlet: postgresql15-server-15.16-1.amzn2023.0.1.x86_64
  Verifying      : libicu-67.1-7.amzn2023.0.4.x86_64
  Verifying      : postgresql15-15.16-1.amzn2023.0.1.x86_64
  Verifying      : postgresql15-private-libs-15.16-1.amzn2023.0.1.x86_64
  Verifying      : postgresql15-server-15.16-1.amzn2023.0.1.x86_64

Installed:
libicu-67.1-7.amzn2023.0.4.x86_64                postgresql15-15.16-1.amzn2023.0.1.x86_64
postgresql15-private-libs-15.16-1.amzn2023.0.1.x86_64  postgresql15-server-15.16-1.amzn2023.0.1.x86_64

Complete!
[ec2-user@ip-172-31-58-166 kafka_2.13-4.2.0]$ sudo postgresql-setup --initdb
* Initializing database in '/var/lib/pgsql/data'
* Initialized, logs are in /var/lib/pgsql/initdb_postgresql.log
[ec2-user@ip-172-31-58-166 kafka_2.13-4.2.0]$ sudo systemctl start postgresql
sudo systemctl enable postgresql
Created symlink /etc/systemd/system/multi-user.target.wants/postgresql.service → /usr/lib/systemd/system/postgresql.service.
[ec2-user@ip-172-31-58-166 kafka_2.13-4.2.0]$ |
```

2. 데이터베이스 및 JSONB 테이블 만들기 (EC2 창)

DB 안으로 관리자 계정으로 걸어 들어가서, 배터리 데이터를 영구
저장할 bms_db와 테이블을 만들기

```
ice.  
[ec2-user@ip-172-31-58-166 kafka_2.13-4.2.0]$ # 1. PostgreSQL 관리자 계정으로 접속  
sudo -i -u postgres psql  
psql (15.16)  
Type "help" for help.  
  
postgres=# CREATE DATABASE bms_db;  
CREATE DATABASE  
postgres=# \c bms_db;  
You are now connected to database "bms_db" as user "postgres".  
bms_db=# CREATE TABLE battery_logs (  
    id SERIAL PRIMARY KEY,  
    data JSONB,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
CREATE TABLE  
bms_db=# \q
```

3.EC2 내부에 파이썬 라이브러리 설치 (EC2 창)

카프카 토픽에서 데이터를 꺼내와 방금 만든 PostgreSQL로 집어넣어 줄
파이썬 컨슈머를 EC2 내부에서 돌릴 예정

이후 파이썬 컨슈머 스크립트 작성 및 대기

4.파이썬 컨수머 스크립트 작성 및 대기

```
import json
import psycopg2
from kafka import KafkaConsumer

KAFKA_SERVER = 'localhost:9092'
TOPIC_NAME = 'battery-data'

# PostgreSQL 접속 정보 (로컬 내부 접속이므로 비번은 비워둡니다)
DB_CONFIG = {
    'host': 'localhost',
    'database': 'bms_db',
    'user': 'postgres',
    'password': ''
}

try:
    # 1. DB 연결
    db_conn = psycopg2.connect(**DB_CONFIG)
    db_cursor = db_conn.cursor()
    print("🐘 Connected to PostgreSQL successfully!")

    # 2. 카프카 컨수머 생성 (최신 데이터부터 polling)
    consumer = KafkaConsumer(
        TOPIC_NAME,
        bootstrap_servers=[KAFKA_SERVER],
        value_deserializer=lambda v: json.loads(v.decode('utf-8')),
```

PART 4. 수집 -> 전달->저장

with vs code(python), kafka, postgresql

1. MCU 더미 데이터 → Kafka로 쓰기 (프로듀서 = MCU 가상 센서 데이터 생성 스크립트)

```
C:\Users\김근우\AppData\Local\Programs\Python\Python314\python.exe: can't open file 'C:\\Users\\김근우\\battery_producer.py': [Errno 2] No such file or directory
PS C:\Users\김근우> python "C:\Users\김근우\Downloads\battery_producer.py"
Successfully connected to Kafka Broker at 3.38.137.121:9092
Starting virtual battery sensor data stream... (Press Ctrl+C to stop)
[Sent] {'timestamp': 1779054998809, 'voltage': 4.196, 'current': -1.186, 'temperature': 25.05, 'soc': 99.96}
[Sent] {'timestamp': 1779054999953, 'voltage': 4.195, 'current': -1.4, 'temperature': 24.81, 'soc': 99.92}
[Sent] {'timestamp': 1779055000954, 'voltage': 4.191, 'current': -0.563, 'temperature': 24.81, 'soc': 99.9}
[Sent] {'timestamp': 1779055001955, 'voltage': 4.188, 'current': -1.505, 'temperature': 24.81, 'soc': 99.88}
[Sent] {'timestamp': 1779055002956, 'voltage': 4.186, 'current': -1.276, 'temperature': 24.9, 'soc': 99.84}
[Sent] {'timestamp': 1779055003957, 'voltage': 4.183, 'current': -1.81, 'temperature': 25.0, 'soc': 99.81}
[Sent] {'timestamp': 1779055004959, 'voltage': 4.178, 'current': -0.639, 'temperature': 24.9, 'soc': 99.8}
[Sent] {'timestamp': 1779055005960, 'voltage': 4.175, 'current': -1.654, 'temperature': 24.96, 'soc': 99.77}
[Sent] {'timestamp': 1779055006962, 'voltage': 4.172, 'current': -1.575, 'temperature': 25.1, 'soc': 99.75}
[Sent] {'timestamp': 1779055007964, 'voltage': 4.171, 'current': -0.924, 'temperature': 25.06, 'soc': 99.7}
[Sent] {'timestamp': 1779055008965, 'voltage': 4.166, 'current': -1.459, 'temperature': 25.0, 'soc': 99.68}
[Sent] {'timestamp': 1779055009967, 'voltage': 4.162, 'current': -1.44, 'temperature': 24.86, 'soc': 99.64}
[ec2-user@ip-172-31-58-166 kafka_2.13-4.2.0]$ bin/kafka-console-consumer.sh --topic battery-data --from-beginning --bootstrap-server localhost:9092
{"timestamp": 1779054998809, "voltage": 4.196, "current": -1.186, "temperature": 25.05, "soc": 99.96}
{"timestamp": 1779054999953, "voltage": 4.195, "current": -1.4, "temperature": 24.81, "soc": 99.92}
{"timestamp": 1779055000954, "voltage": 4.191, "current": -0.563, "temperature": 24.81, "soc": 99.9}
{"timestamp": 1779055001955, "voltage": 4.188, "current": -1.505, "temperature": 24.81, "soc": 99.88}
{"timestamp": 1779055002956, "voltage": 4.186, "current": -1.276, "temperature": 24.9, "soc": 99.84}
{"timestamp": 1779055003957, "voltage": 4.183, "current": -1.81, "temperature": 25.0, "soc": 99.81}
{"timestamp": 1779055004959, "voltage": 4.178, "current": -0.639, "temperature": 24.9, "soc": 99.8}
{"timestamp": 1779055005960, "voltage": 4.175, "current": -1.654, "temperature": 24.96, "soc": 99.77}
{"timestamp": 1779055006962, "voltage": 4.172, "current": -1.575, "temperature": 25.1, "soc": 99.75}
{"timestamp": 1779055007964, "voltage": 4.171, "current": -0.924, "temperature": 25.06, "soc": 99.7}
{"timestamp": 1779055008965, "voltage": 4.166, "current": -1.459, "temperature": 25.0, "soc": 99.68}
{"timestamp": 1779055009967, "voltage": 4.162, "current": -1.44, "temperature": 24.86, "soc": 99.64}
{"timestamp": 1779055010968, "voltage": 4.158, "current": -0.845, "temperature": 25.16, "soc": 99.61}
{"timestamp": 1779055011969, "voltage": 4.154, "current": -1.707, "temperature": 24.89, "soc": 99.59}
{"timestamp": 1779055012971, "voltage": 4.153, "current": -1.271, "temperature": 25.07, "soc": 99.58}
{"timestamp": 1779055013973, "voltage": 4.149, "current": -0.874, "temperature": 25.01, "soc": 99.57}
{"timestamp": 1779055014980, "voltage": 4.148, "current": -0.674, "temperature": 25.13, "soc": 99.54}
{"timestamp": 1779055015982, "voltage": 4.146, "current": -0.673, "temperature": 25.21, "soc": 99.52}
{"timestamp": 1779055016984, "voltage": 4.141, "current": -1.982, "temperature": 25.03, "soc": 99.51}
{"timestamp": 1779055017986, "voltage": 4.14, "current": -1.228, "temperature": 25.22, "soc": 99.46}
{"timestamp": 1779055018987, "voltage": 4.136, "current": -1.55, "temperature": 25.19, "soc": 99.42}
{"timestamp": 1779055019994, "voltage": 4.131, "current": -1.3, "temperature": 25.05, "soc": 99.38}
{"timestamp": 1779055021009, "voltage": 4.128, "current": -0.748, "temperature": 25.18, "soc": 99.36}
{"timestamp": 1779055022019, "voltage": 4.124, "current": -0.814, "temperature": 24.88, "soc": 99.34}
{"timestamp": 1779055023012, "voltage": 4.119, "current": -1.716, "temperature": 25.02, "soc": 99.32}
```

2. PostgreSQL 설치하고 파이썬 컨슈머로 연동해서 DB에 영구 저장

```
[ec2-user@ip-172-31-58-166 kafka_2.13-4.2.0]$ sudo systemctl restart postgresql
```

```
[ec2-user@ip-172-31-58-166 kafka_2.13-4.2.0]$ python3 battery_consumer.py
```

```
Connected to PostgreSQL successfully!
```

```
Waiting for battery data from Kafka topic 'battery-data'...
```

```
[DB Saved] TS: 1779057200472 | Vol: 4.196V | SOC: 99.98%
```

```
[DB Saved] TS: 1779057201615 | Vol: 4.192V | SOC: 99.96%
```

```
[DB Saved] TS: 1779057202617 | Vol: 4.189V | SOC: 99.93%
```

```
[DB Saved] TS: 1779057203618 | Vol: 4.184V | SOC: 99.89%
```

```
[DB Saved] TS: 1779057204619 | Vol: 4.181V | SOC: 99.86%
```

```
[DB Saved] TS: 1779057205620 | Vol: 4.178V | SOC: 99.83%
```

```
[DB Saved] TS: 1779057206622 | Vol: 4.174V | SOC: 99.79%
```

```
[DB Saved] TS: 1779057207624 | Vol: 4.169V | SOC: 99.76%
```

```
[DB Saved] TS: 1779057208625 | Vol: 4.166V | SOC: 99.71%
```

```
[DB Saved] TS: 1779057209633 | Vol: 4.163V | SOC: 99.66%
```

```
[DB Saved] TS: 1779057210635 | Vol: 4.162V | SOC: 99.65%
```

```
[DB Saved] TS: 1779057211639 | Vol: 4.159V | SOC: 99.61%
```

```
[DB Saved] TS: 1779057212641 | Vol: 4.155V | SOC: 99.56%
```

```
[DB Saved] TS: 1779057213642 | Vol: 4.154V | SOC: 99.52%
```

```
[DB Saved] TS: 1779057214643 | Vol: 4.153V | SOC: 99.49%
```

```
[DB Saved] TS: 1779057215645 | Vol: 4.148V | SOC: 99.44%
```

```
[DB Saved] TS: 1779057216646 | Vol: 4.146V | SOC: 99.39%
```

```
[DB Saved] TS: 1779057217648 | Vol: 4.145V | SOC: 99.35%
```

```
[DB Saved] TS: 1779057218649 | Vol: 4.14V | SOC: 99.32%
```

```
[DB Saved] TS: 1779057219650 | Vol: 4.136V | SOC: 99.27%
```

```
[DB Saved] TS: 1779057220652 | Vol: 4.135V | SOC: 99.23%
```

```
[DB Saved] TS: 1779057221653 | Vol: 4.133V | SOC: 99.18%
```

```
[DB Saved] TS: 1779057222655 | Vol: 4.131V | SOC: 99.14%
```

```
[DB Saved] TS: 1779057223656 | Vol: 4.126V | SOC: 99.1%
```

```
[DB Saved] TS: 1779057224657 | Vol: 4.122V | SOC: 99.07%
```

결론

- 컨슈머 프로그램을 종료하더라도, 이미 카프카를 거쳐 PostgreSQL 데이터베이스 장부에 저장된 배터리 데이터들은 유실되지 않고 아주 안전하게 보관되어 있으니